南开大学

高级语言程序设计

大作业实验报告

Claw++ – AI Agent 桌面应用

https://github.com/DavidLi-TJ/ClawPP-Agent

姓名: 李厚贤

学号: 2510669

开发软件: Visual Studio Code / MinGW

目录

1 作业题目 **3**

2 开发软件 **3**

3 课题要求 **3**

3.1 面向对象设计 3

3.2 核心技术 4

3.3 创新点 4

4 项目演示 **5**

4.1 主界面 5

4.2 模型配置界面 6

5 主要流程 **6**

5.1 整体架构 6

5.2 核心模块实现 7

5.2.1 ReAct 推理引擎 7

5.2.2 双层上下文压缩 8

5.2.3 工具调用系统 8

5.2.4 三级权限控制与 BashAnalyzer 8

5.2.5 流式对话（SSE 解析） 9

5.2.6 系统提示词构建（ContextBuilder） 9

5.2.7 技能插件系统 9

5.2.8 QmlBackend – QML 与 C++ 桥梁 9

5.2.9 多线程与会话管理 9

5.2.10 23 种模型服务商 10

5.2.11 数据库 10

5.3 构建与打包 10

5.4 单元测试 10

6 难点与解决 **11**

6.1 流式 ReAct 循环 11

6.2 上下文窗口管理 11

6.3 多线程会话管理 11

6.4 QML 与 C++ 集成 11

6.5 SSE 数据不完整性 11

7 收获 **11**

8 AI 工具使用说明 **12**

9 项目仓库 **12**

10 总结 **12**

1 作业题目

基于 Qt/C++ 的现代化 AI Agent 桌面应用程序 – **Claw++**

本项目实现了一个完整的 AI 智能代理桌面应用，采用 Qt/C++ 框架开发，具备图形化用户界面，支持多模型服务商接入、ReAct 推理引擎、工具调用系统、智能记忆管理等功能。项目采用 C++17 标准编写，使用 Qt 6.5+ 构建 Windows 11 风格玻璃质感界面。

核心特性：

- ReAct 推理引擎：思考-行动-观察循环，支持多轮自主推理与工具调用
- 9 大内置工具：文件读写、Shell 命令、网络请求、子代理、搜索等
- 实时流式对话：基于 SSE 的流式响应
- 三级权限系统：Safe/Moderate/Dangerous，Shell 命令风险评分
- 双层上下文压缩：MicroCompact + Full Compact + CompactState 持续追踪
- 技能插件系统：Markdown 格式技能定义，运行时热加载
- 23 种模型服务商：覆盖国内外主流大模型平台
- SQLite 持久化：会话、消息、记忆全量存储

2 开发软件

表 1: 开发环境与工具

类别	**工具/框架**	**版本**
编程语言	C++	C++17
GUI 框架	Qt Framework	6.5+
构建系统	CMake	3.16+
数据库	SQLite	3.x
开发环境	Visual Studio / Qt Creator	2019+
版本控制	Git	2.x
安装打包	Qt Installer Framework	4.x

3 课题要求

3.1 面向对象设计

- 抽象接口：`IAgentCore`、`ILLMProvider`、`ITool`、`IMemorySystem`
- 多态实现：通过虚函数实现不同 Provider 的统一调用

- 单例模式：ToolRegistry、PermissionManager、ConfigManager、ContextBuilder
- 工厂模式：ProviderManager 统一管理多种模型服务商

3.2 核心技术

- 事件驱动：Qt 信号-槽机制的松耦合通信
- 多线程：会话隔离，每个会话独立线程处理
- 数据持久化：SQLite 存储会话和消息
- 网络通信：HTTP RESTful API + SSE 流式响应

3.3 创新点

- ReAct 推理循环的完整工程实现
- 双层上下文压缩的完整工程实现
- Bash 命令安全分析器（风险评分 1-4 级）
- 基于 Markdown 的可扩展技能插件
- Windows 11 玻璃质感 UI（毛玻璃、液态按钮、涟漪效果）

4 项目演示

4.1 主界面

图 1: Claw++ 主界面 – Windows 11 风格玻璃质感聊天视图

主界面左侧为会话列表面板（新建、重命名、删除、置顶），中间为聊天视图。采用 Qt Quick (QML) 构建 Windows 11 Fluent Design 风格：毛玻璃面板、液态按钮、涟漪效果、可自定义背景和行距。

4.2 模型配置界面

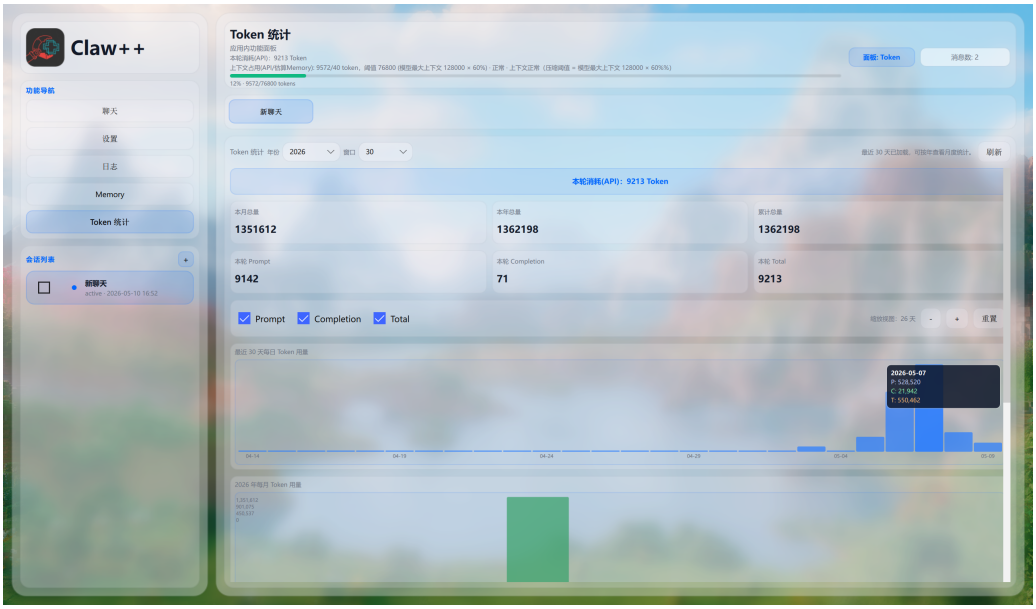

图 2: 模型配置界面 – 支持 23 种服务商

支持配置 API Key、Base URL、模型选择，覆盖 OpenAI、Anthropic、Gemini、DeepSeek、智谱 AI 等 23 种服务商。配置数据由 `ConfigManager` 单例管理，持久化存储于 `~/.clawpp/config.json`。

5 主要流程

5.1 整体架构

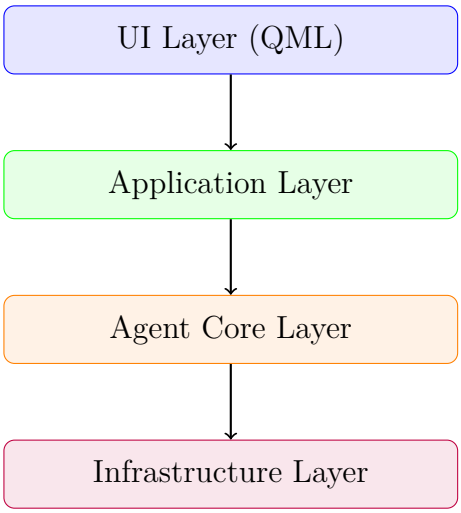

图 3: 四层架构设计图

表 2: 架构层级职责

层级	关键类
UI 层	QmlBackend、SessionListModel、MessageListModel、ChatView
应用层	AgentService、SessionManager、SessionThread、TemplateLoader
核心层	ReactAgentCore、IAgentCore、ContextBuilder
基础设施层	Provider、Memory、Tool、Permission、Database、Config、Logger、Event

5.2 核心模块实现

5.2.1 ReAct 推理引擎

算法流程：

1. ContextBuilder 构建系统提示词（身份、技能、工具、记忆）
2. 调用 LLM 生成思考，流式输出
3. 流中实时解析工具调用 JSON
4. ToolExecutor 执行工具
5. 收集结果追加到上下文
6. 继续迭代或终止

流式工具预执行：SSE 流中检测到工具调用后立即启动预执行，使用 `QMap<QString, QFuture<ToolResult>>` 并发执行，结果缓存到流完成时直接使用。

7 种终止策略：

表 3: ReAct 循环终止策略

策略	说明
NormalCompletion	正常完成，生成最终答案
MaxIterationsReached	达到最大迭代次数（25 轮）
NoToolStreak	连续无工具调用（3 轮）
ContextOverflow	上下文超限，触发压缩
ToolExecutionFailure	工具失败，注入恢复消息重试
UserStop	用户主动停止
StreamError	流式失败，按重试计数处理

5.2.2 双层上下文压缩

表 4: 上下文压缩策略

层级	策略	操作
Micro	MicroCompact	将旧工具结果替换为占位摘要，最新工具结果永不折叠
Full	Full Compact	存档完整消息、调模型生成结构化摘要、用摘要替换旧历史

压缩触发:上下文超过阈值时先执行 MicroCompact,若仍超限则执行 Full Compact。

MemoryManager 长期记忆： 跨会话记忆落盘到 `memory/` 目录，`MEMORY.md` 存储用户偏好，`HISTORY.md` 记录会话摘要。

5.2.3 工具调用系统

表 5: 10 大内置工具

工具	功能	权限
read_file	读取文件	Safe
write_file	写入文件	Moderate
edit_file	搜索替换编辑	Moderate
list_directory	列出目录	Safe
search_files	搜索文件	Safe
shell_tool	执行 Shell 命令	Dangerous
network_tool	HTTP 请求	Moderate
subagent_tool	调用子代理	Moderate
system_tool	系统信息查询	Safe
compact_tool	手动触发上下文压缩	Moderate

执行流程： 解析工具 JSON → 检查权限 → ToolExecutor 执行 → 返回 ToolResult → ContextBuilder 加入上下文 → 下一轮迭代。

5.2.4 三级权限控制与 BashAnalyzer

权限判断流程：白名单检查 → 规则模式匹配 → 返回权限结果（Auto-Allow/Ask-User/Auto-Deny）→ QCache 缓存。

Shell 命令通过 `BashAnalyzer` 进行 8 维度安全检查：

表 6: BashAnalyzer 安全检查

维度	**检测内容**	**风险**
危险命令	rm -rf, mkfs, dd	Critical
命令注入	$(...), 反引号, 管道	High
敏感路径	/etc/shadow, /root/	High
Glob 炸弹	*{1..1000000}	Medium
重定向	> /dev/sda	Critical
后台进程	nohup, &	Medium
网络操作	curl, wget	Medium
提权	sudo, su	High

风险等级取所有安全问题中的最大值。

5.2.5 流式对话（SSE 解析）

QNetworkReply 接收数据 → 追加缓冲区 → 按 data: 分割 → 解析 JSON chunk → 提取 content/tool_calls/usage → 遇到 [DONE] 完成。内部缓冲区处理不完整事件。

5.2.6 系统提示词构建（ContextBuilder）

组装流程：SOUL.md（身份）→ AGENTS.md（规则）→ TOOLS.md（工具）→ USER.md（偏好）→ MEMORY.md（记忆）→ 技能列表 → 工具 Schema → 完整 prompt。

5.2.7 技能插件系统

技能通过 Markdown 文件定义，包含 YAML 元数据、触发词、系统提示、所需工具。加载时扫描目录、解析 frontmatter、检查依赖。用户输入时匹配触发词激活。

5.2.8 QmlBackend – QML 与 C++ 桥梁

通过 `Q_PROPERTY` 暴露属性，`Q_INVOKABLE` 暴露方法，`SessionListModel`/`MessageListModel` 提供列表数据，信号-槽实现双向绑定。

5.2.9 多线程与会话管理

每个会话由独立 `SessionThread` 承载，AgentService 通过 `moveToThread()` 迁移，UI 通过 `QueuedConnection` 通信，AgentOrchestrator 统一管理生命周期。

5.2.10 23 种模型服务商

表 7: 模型服务商

类别	服务商
国际	OpenAI, Anthropic, Gemini, Mistral, Groq, OpenRouter, GitHub Copilot, Azure, Codex
国内	DeepSeek, Z.ai, 月之暗面, 通义, 硅基流动, 阶跃星辰, MiniMax, 火山, BytePlus
本地	Ollama, OpenVINO, vLLM

ProviderManager 工厂模式，所有 Provider 实现 `ILLMProvider` 统一接口。

5.2.11 数据库

SQLite 存储 sessions 表（id/name/created_at/updated_at）和 conversations 表（id/session_id/role/content/tool_calls/created_at）。

5.3 构建与打包

CMake 构建，C++17，Qt6 组件（Core/Gui/Qml/Quick/Network/Sql/Widgets）。IFW 打包为安装包（约 57.7 MB），`build_installer.bat` 自动完成 Release → 收集 → 打包。

5.4 单元测试

表 8: 核心测试案例

模块	测试内容	预期结果
ReAct 循环	思考-行动-观察流程	正确执行工具
SSE 解析	标准 SSE 数据流	正确解析 chunk
权限控制	三级工具调用	按级别返回结果
记忆压缩	超阈值上下文	Token 降至阈值下
BashAnalyzer	危险 Shell 命令	正确识别风险
Provider 切换	切换服务商	正确调用 API

6 难点与解决

6.1 流式 ReAct 循环

传统实现等待完全响应后处理工具，流式模式下工具信息可能在流中间。解决：CorePendingToolCall 缓存流式解析结果，QFuture 并发预执行工具，m_runGeneration 代际标识屏蔽过期回调。

6.2 上下文窗口管理

多轮对话超出 LLM 限制。解决：双层压缩管道，MicroCompact 快速清理旧工具结果，Full Compact 调用 LLM 生成结构化摘要，实时 Token 估算触发压缩。

6.3 多线程会话管理

多个会话独立运行不阻塞。解决：SessionThread 独立承载，moveToThread 迁移对象，QueuedConnection 跨线程通信，AgentOrchestrator 统一管理。

6.4 QML 与 C++ 集成

前端需要实时获取后端流式数据。解决：QmlBackend 桥梁，Q_PROPERTY 暴露属性，QAbstractListModel 提供列表数据，Q_INVOKABLE 暴露方法。

6.5 SSE 数据不完整性

网络传输中事件可能被截断。解决：内部缓冲区累积数据，只处理完整 data: 行，不完整行保留等待下次到达。

7 收获

- **C++**：虚函数多态、模板编程（Result<T>）、Lambda、Qt 父子关系内存管理
- **Qt**：信号-槽事件驱动、MOC 元对象系统、QML/C++ 混合编程、QNetworkAccessManager
- **软件工程**：分层架构、5 种设计模式（单例/工厂/策略/观察者/门面）、Git 版本控制、模块化开发
- **AI 工程化**：ReAct 范式、工具调用流程、上下文压缩、Prompt Engineering、安全风险评估

8 AI 工具使用说明

表 9: AI 工具使用情况

工具	用途	使用位置
Claude Code	UI 界面开发、美化	整体开发
Codex	代码审查、文档撰写	代码优化、报告

AI 工具辅助 UI 部分、数据库部分、上下文压缩的代码生成和文档撰写，其余工具调用、API 调用、ReAct 循环等核心逻辑和架构由开发者自主完成。

9 项目仓库

- GitHub：`https://github.com/DavidLi-TJ/ClawPP-Agent`
- 语言：C++，QML

10 总结

本项目实现了功能完整的 AI Agent 桌面应用，涵盖 ReAct 推理引擎、多模型支持、工具调用、记忆管理、权限控制等功能。采用 Qt/C++ + QML 构建现代化 UI。

项目规模： 60+ 个 C++ 文件，15+ 个 QML 组件，约 10,000+ 行代码，5+ 种设计模式，23 种模型服务商，9 种工具，6 个模板文件。

通过本项目深入理解了面向对象设计、设计模式、多线程、网络通信等核心技术。